El proceso exacto.
Para Claude.

SOP DE ANUNCIOS · INSTALACIÓN Y EJECUCIÓN

Chrome + TikTok/Pinterest + SSSTik
Kie REST + modelos y payloads
Voz, música, HyperFrames y QC literal

Versión 2026-10-10 v2
Incluye scripts instalables, carpetas, comandos y las recetas completas del SOP vigente.

Usar con el paquete Claude adjunto. El PDF guía la instalación; no contiene claves ni sesiones. El foco es ejecutar la fase de anuncios dentro de modeler/modelar, conservando el proceso existente.

SOP exacto para Claude: de referencia a anuncio

Versión 2026-10-10 v2. Instalación y ejecución local con Claude Code + Claude in Chrome. El flujo también puede ejecutarse en Cowork local cuando dispone de las mismas herramientas de Chrome, archivos y terminal. No está diseñado para un chat de Claude sin acceso al equipo.

01. Stack exacto y qué se instala

Agente: Claude Code. Navegador elegido: Google Chrome de la persona, con la extensión oficial Claude in Chrome conectada y su sesión de TikTok/Pinterest/Envato. No abrir un perfil automatizado sin sesión ni sustituirlo silenciosamente por otro navegador. Descarga TikTok: https://ssstik.io/es en una pestaña de ese mismo Chrome. Alternativa de descarga: yt-dlp, cuando el sitio o su descarga no funciona. Pinterest se usa para localizar el video y su fuente original; no se le aplica SSSTik porque no es un descargador universal. Generación: REST de Kie.ai. Imágenes: gpt-image-2-text-to-image y gpt-image-2-image-to-image, resolución 1K. Video de b-roll desde imagen: bytedance/v1-pro-fast-image-to-video, 720p, 5s; 10s únicamente si la acción lo requiere. Voz paga: elevenlabs/text-to-speech-multilingual-v2 vía Kie. UGC hablado: foto propia generada + voz + kling/ai-avatar-standard; pro cuando standard no da el acabado necesario y dentro de presupuesto aprobado. Composición: HyperFrames. Preparación/QC: FFmpeg/ffprobe, Python y Whisper local para transcribir. DALL-E no es el modelo de este proceso. No enviar model=DALL-E ni usar Nano Banana. El identificador que Claude debe enviar para imágenes es gpt-image-2-text-to-image, o su variante image-to-image con referencias. No confundir generar un retrato con generar el video hablando. Estos modelos fijan una receta reproducible basada en el stack documentado y producciones existentes. No se certifica que sean siempre los más baratos: antes de gastar comparar tarifas actuales y cambiar solo con justificación y dentro de la autorización. No trasladar precios históricos de Veo a otro modelo. Fuentes: memorias de imagen GPT del 2026-09-14; producción Cartón y Circuitos del 2026-10-10; WIKI/kie-ai-generacion.md y documentación oficial enlazada al final.

02. Instalar Claude y conectar Chrome

En macOS/Linux, usar el instalador oficial actual; ejemplo de instalación publicado por Anthropic:

```
curl -fsSL https://claude.ai/install.sh | bash
claude --version
```

En Windows nativo, usar PowerShell según documentación oficial; ejemplo:

```
irm https://claude.ai/install.ps1 | iex
claude --version
```

Abrir Google Chrome e instalar Claude in Chrome desde el enlace oficial de la documentación de Anthropic, iniciar sesión y dejar Chrome abierto. Iniciar Claude Code y autenticarse con /login con una cuenta compatible. Conectar Kie como API de generación no reemplaza el login de Claude; la integración Chrome requiere su propia autenticación compatible. Para este proceso iniciar en terminal:

```
claude --chrome
```

Dentro de Claude ejecutar /chrome y verificar conexión de la extensión. En Cowork, conectar Claude in Chrome en la sesión y comprobar que están presentes las herramientas mcp__claude-in-chrome__*. Autorizar los sitios en los diálogos de la plataforma cuando corresponda. Si aparece login/CAPTCHA en TikTok, lo resuelve la persona; Claude reanuda después. No copiar cookies ni contraseñas al manual. Instalar Node.js LTS y Python 3 desde sus distribuidores oficiales si faltan. En macOS con Homebrew ya disponible:

```
brew install node python ffmpeg yt-dlp
python3 -m venv "$HOME/Renovaciones-local/.venv-ads"
"$HOME/Renovaciones-local/.venv-ads/bin/pip" install requests python-dotenv openai-whisper
npx hyperframes skills
npx hyperframes --version
```

En Windows usar Node/Python/FFmpeg nativos y Claude fuera de WSL para la integración Chrome. Instalar los mismos paquetes en un venv; activar con .venv-ads\Scripts\Activate.ps1. Si un gestor no está disponible, seguir la instalación oficial de esa dependencia; no fingir que brew existe. En cada equipo guardar las versiones de claude, node, Python, ffmpeg, yt-dlp y HyperFrames en logs/versiones.txt. Para reproducir un proyecto futuro, conservar package-lock.json y la versión de HyperFrames usada. La instalación requiere cuenta/acceso y configuración una vez. El paquete no incluye claves, sesiones ni créditos de otra persona.

03. Instalar las skills y conservar modeler/modelar

Descomprimir paquete-claude.zip. Dentro están instalar.py, ad-creator/, watch/ y modeler/. Elegir el ámbito: ~/.claude/skills para todas las sesiones o <proyecto>/.claude/skills solo para ese proyecto. La carpeta lleva SKILL.md con frontmatter válido; no alcanza con dejar el PDF suelto en una carpeta. Desde la carpeta descomprimida:

```
python3 instalar.py --dry-run
python3 instalar.py
```

Para ámbito del proyecto:

```
python3 instalar.py --target "/ruta/proyecto/.claude/skills"
```

El instalador respalda archivos existentes en ~/.claude/respaldos-sop-anuncios/<fecha>/, o en el padre del destino equivalente. Preserva el SKILL.md existente y añade un bloque que obliga a leer el contrato exacto. Si ya existe modeler lo integra allí; si no, usa modelar existente; si no existe ninguno crea modeler mínimo de creativos. No sustituye fases de oferta, tienda ni lanzamiento. Imprime cada archivo tocado y la ruta de respaldo. Reiniciar la sesión Claude si las skills nuevas no aparecen. Pedir: «Lee /watch, /ad-creator y mi modeler/modelar. Dime qué archivo obliga a usar Chrome + SSSTik y el cliente Kie. Ejecuta solo el diagnóstico sin gasto». Revisar que no esté cargando otra skill de proyecto con el mismo nombre que oculte la instalación personal.

04. Carpetas exactas de trabajo

Raíz de producción: ~/Renovaciones-local/<AAAA-MM-DD-oferta-mercado>. Mantenerla fuera de iCloud/OneDrive y no reutilizar un slug de otra oferta. Instalar las skills fuera de esta carpeta: son instrucciones, no archivos de render. Ejecutar desde terminal o Bash de Claude:

```
python3 "$HOME/.claude/skills/ad-creator/scripts/iniciar-proyecto.py" 2026-10-10-papa-es
```

El script crea este árbol y los CSV con sus encabezados:

```
~/Renovaciones-local/2026-10-10-papa-es/
  00-brief/                     oferta, mercado, decisiones
  01-referencias/originales/     ad01-original.mp4
  01-referencias/frames/         frames por clip/transición
  01-referencias/audio/          original.wav y transcripción
  02-watch/                     informe.md y clips.csv
  03-busquedas/                 busquedas.csv y evidencias
  04-assets/descargados/tiktok/  C001-tiktok-<id>.mp4
  04-assets/descargados/pinterest/
  04-assets/stock/              clips licenciados
  04-assets/generados/imagenes/ C001.png
  04-assets/generados/video/    C001.mp4
  04-assets/ugc/                avatar y clips hablados
```

```
  04-assets/manifest.csv        procedencia y QC
  05-audio/voz/                 voz-es.mp3, voz-es.srt
  05-audio/musica/              cama.mp3, licencia.txt
  05-audio/sfx/                 sonidos seleccionados
  06-hyperframes/ad01/          proyecto editable
  07-renders/                  ad01-v01.mp4 ... ad06.mp4
  08-qc/                       qc.md, correspondencias.csv
  09-entrega/                  preview.html, copy.md
  logs/kie/                    upload, task y estado JSON
  prompts/                     imagen, video, voz, ugc JSON
  presupuestos/                ad01, ads02-06 y aprobaciones
```

Para comandos siguientes definir estas variables y usar siempre las comillas:

```
export ADS_PROJECT="$HOME/Renovaciones-local/2026-10-10-papa-es"
export ADS_SKILL="$HOME/.claude/skills/ad-creator"
export ADS_PYTHON="$HOME/Renovaciones-local/.venv-ads/bin/python"
cd "$ADS_PROJECT"
```

Si se eligió ámbito de proyecto, ADS_SKILL apunta a esa instalación. Nombres por clip C001, C002, etc.; por anuncio ad01 ... ad06; por revisión v01, v02. No sobreescribir originales. Los recursos compartidos viven una sola vez en 04-assets; los proyectos enlazan o usan la copia mínima que necesite HyperFrames.

05. Watch: descargar referencia, ver todos los clips y escuchar

Abrir la referencia en Chrome mediante las herramientas de la sección siguiente; guardar URL, ID y duración en 00-brief/referencia.md. Para una URL compatible se puede descargar el original con yt-dlp:

```
yt-dlp --no-playlist -f "bv*+ba/b" --merge-output-format mp4 \
  -o "$ADS_PROJECT/01-referencias/originales/ad01-original.%(ext)s" \
  "URL_DEL_VIDEO"
```

Para TikTok también se permite SSSTik, siguiendo el paso 07. En Ad Library mantener el orden visible del arsenal, sin ranking nuevo. Si no se puede descargar, observar reproducción por Chrome y documentar la limitación. No declarar que se vio un video solo por caption o transcripción. Preparar audio y evidencia visual:

```
ffmpeg -i "$ADS_PROJECT/01-referencias/originales/ad01-original.mp4" \
  -vn -ac 1 -ar 16000 "$ADS_PROJECT/01-referencias/audio/original.wav"
ffmpeg -i "$ADS_PROJECT/01-referencias/originales/ad01-original.mp4" \
  -vf "select='gt(scene,0.25)',showinfo" -fps_mode vfr \
  "$ADS_PROJECT/01-referencias/frames/corte-%04d.jpg"
"$HOME/Renovaciones-local/.venv-ads/bin/whisper" \
  "$ADS_PROJECT/01-referencias/audio/original.wav" \
  --model small --output_format all \
  --output_dir "$ADS_PROJECT/01-referencias/audio"
```

El detector de escena solo propone cortes: Watch revisa todos los segmentos del video en movimiento, incluidos insertos que el umbral omitió, transiciones y formatos simultáneos. Para revisar un tramo de 2.4s desde 1.2s:

```
ffmpeg -ss 1.2 -i "$ADS_PROJECT/01-referencias/originales/ad01-original.mp4" \
  -t 2.4 -c:v libx264 -c:a aac \
  "$ADS_PROJECT/01-referencias/originales/revision-C001.mp4"
```

Ver ese MP4 con el mecanismo audiovisual disponible en Claude; imágenes aisladas no acreditan movimiento ni escuchar. Whisper transcribe, no certifica música/SFX/acento. Si la sesión no puede oír el audio, declarar audio pendiente y resolverlo antes de afirmar QC completo. Guardar 02-watch/informe.md y clips.csv. Por clip: ID, inicio/fin, frase, formato/capas, objeto literal, acción por fases, encuadre, cámara, luz, subtítulos, popups, transiciones, música/SFX y función. Cubrir todo el tiempo, permitiendo capas superpuestas. Ejemplo: C001, 0.00-1.80, papa cruda en mano girando sobre mesa, macro vertical, luz cálida; imprescindible papa + giro + mano. No sustituir por una ensalada. Watch entrega este mapa a ad-creator. Conserva guion íntegro y orden de

argumentos; traducción localizada completa. Claims ajenos se ajustan a evidencia real de la oferta.

06. Cómo Claude entra y busca en TikTok/Pinterest

Antes de usar el navegador, Claude lee su skill de Chrome. Si los tools son diferidos, cargar en una sola ToolSearch los necesarios:

```
select:mcp__claude-in-chrome__tabs_context_mcp,
mcp__claude-in-chrome__tabs_create_mcp,
mcp__claude-in-chrome__navigate,
mcp__claude-in-chrome__read_page,
mcp__claude-in-chrome__computer,
mcp__claude-in-chrome__form_input
```

La query real se envía en una línea. Si solo existe enable__mcp__claude-in-chrome, activarlo primero y después descubrir los tools. Los parámetros se toman del schema expuesto en esa instalación; no inventar tabId, ref ni coordenadas.

1. Llamar tabs_context_mcp primero. Crear una pestaña propia con tabs_create_mcp y guardar su tabId. Reutilizar una pestaña de la persona solo si ella pidió trabajar allí.

2. Con navigate abrir https://www.tiktok.com/ en esa pestaña. Usar read_page y screenshot/computer para identificar el buscador real. Con form_input introducir la consulta; enviar Enter con computer. Alternativa de navegación: https://www.tiktok.com/search?q=<consulta-codificada>. Si esa ruta cambia, usar el buscador visible.

3. Por cada clip de Watch construir consultas específicas en original, inglés y mercado: objeto + acción + plano. Para C001: «potato hand rotating close up», «papa girando mano primer plano»; no buscar solo «comida».

4. Abrir candidatos, reproducir y comprobar acción/encuadre/duración. Buscar al menos tres variantes de consulta antes de dar un b-roll por ausente, salvo bloqueo de acceso; esta es una regla operativa de esta versión, no un hecho histórico del sistema. Guardar resultados útiles y descartes con motivo en 03-busquedas/busquedas.csv.

5. Abrir la URL propia del video, copiar enlace desde Compartir > Copiar enlace o la dirección confirmada de la publicación. Registrar URL completa, @autor, ID y segundos útiles. No copiar la URL de resultados como si fuera el video.

6. Buscar también en web general y Pinterest antes de generar b-roll. En nueva pestaña abrir https://www.pinterest.com/, usar su buscador y las mismas consultas. Abrir cada pin, reproducir cuando sea video y entrar a «Visitar»/fuente original. Guardar pin y origen. Un pin de foto no resuelve una acción en movimiento.

7. Cuando internet/TikTok/Pinterest no aporten coincidencia adecuada, consultar catálogo propio y Envato. Si siguen faltando clips, planificar generación. UGC necesariamente propio puede ir directamente a planificación IA. No generar toda la tanda sin intentar conseguir b-roll real.

Si Chrome no responde, comprobar /chrome; tras 2-3 intentos fallidos parar esa ruta y comunicar el bloqueo. No afirmar búsqueda hecha. No crear un perfil nuevo, pedir passwords por chat ni saltar CAPTCHA. Reanudar tareas independientes y usar el camino de descarga accesible.

07. Descargar TikTok con SSSTik y verificar dónde quedó

Sitio exacto: https://ssstik.io/es. Es la ruta documentada en recursos anteriores del proyecto. La secuencia se realiza en Chrome con Claude in Chrome; no requiere instalar un supuesto MCP de SSSTik.

1. Mantener la pestaña del TikTok candidato. Crear otra para SSSTik, navigate a la URL exacta y leer la página. Identificar el campo para el enlace por texto/DOM visible; form_input pega la URL de la publicación.

2. Pulsar «Descargar» en el formulario real. Esperar el resultado observando el estado nuevo. Elegir la opción de video MP4 disponible adecuada, sin seleccionar anuncios ni MP3 si se requiere b-roll. Los nombres de botones pueden cambiar: confirmar lo que dice la página en cada ejecución.

3. Si hay anuncios, pestañas externas o pedidos de instalar una extensión, volver al tabId de SSSTik y usar solo el resultado real. No descargar ejecutables. Si falla tras intentos razonables, pasar a yt-dlp o registrar fallo, sin asegurar que SSSTik siempre puede bajar cualquier video.

4. Si Chrome muestra «Guardar como», usar 04-assets/descargados/tiktok/C001-tiktok-<id>.mp4. Si guarda automático, consultar el historial de Chrome en chrome://downloads y el nombre/ubicación de la descarga recién iniciada. La ruta por defecto suele ser ~/Downloads, pero usar la configuración real de ese Chrome. Esperar a que termine; un .crdownload no es MP4 terminado.

5. Claude con acceso local mueve solo ese archivo confirmado a la carpeta del proyecto y lo renombra. No mover «el último MP4» sin comprobar su nombre/origen. Ejemplo después de identificar el archivo exacto:

```
mv "$HOME/Downloads/ARCHIVO_CONFIRMADO.mp4" \
  "$ADS_PROJECT/04-assets/descargados/tiktok/C001-tiktok-ID.mp4"
ffprobe -v error -show_streams -show_format \
  "$ADS_PROJECT/04-assets/descargados/tiktok/C001-tiktok-ID.mp4"
```

6. Reproducirlo: comprobar que es el candidato elegido, que contiene el tramo y que su calidad sirve. Calcular SHA-256 con Python/hashlib o herramienta del sistema; guardar autor/URL/licencia/permisos y QC en manifest.csv. Descargar sin watermark no otorga derechos publicitarios; anotar el estado real y usar material autorizado en la entrega comercial.

Alternativa concreta para URL de TikTok o fuente compatible, con descarga directa a destino:

```
yt-dlp --no-playlist -f "bv*+ba/b" --merge-output-format mp4 \
  -o "$ADS_PROJECT/04-assets/descargados/tiktok/C001-tiktok-%(id)s.%(ext)s" \
  "URL_CONFIRMADA_DEL_TIKTOK"
```

No afirmar que yt-dlp admite todo. Para Pinterest, abrir fuente original y usar descarga del sitio o yt-dlp si es compatible; guardar en descargados/pinterest. Cuando la fuente sea TikTok, descargar mediante SSSTik y mantener ambos enlaces. Recortar una toma útil sin modificar original:

```
ffmpeg -ss 2.0 -i "ARCHIVO_ORIGINAL.mp4" -t 1.8 -an \
  -c:v libx264 -crf 18 "CLIP_C001.mp4"
```

08. Configurar Kie sin exponer claves y comprobar saldo

Crear cuenta en https://kie.ai/ y obtener una API key desde su panel. Guardarla localmente en la carpeta de trabajo como .env.local; la persona introduce el valor en su editor, no lo manda en el PDF ni lo deja en SKILL.md:

```
KIE_API_KEY=valor_privado_de_la_persona
```

Añadir .env.local a .gitignore y restringir permisos cuando corresponda. No usar cat .env.local en salidas compartidas. El script kie.py lee el secreto mediante python-dotenv y os.environ; el header es Authorization: Bearer <clave>. No se incluye la clave en payloads ni logs. Consulta de saldo, sin generación:

```
cd "$ADS_PROJECT"
"$ADS_PYTHON" "$ADS_SKILL/scripts/kie.py" credits
```

Endpoint exacto: GET https://api.kie.ai/api/v1/chat/credit. Éxito requiere code=200 en JSON, además de HTTP correcto. 401: clave inválida; 402: saldo insuficiente; 429: límite de solicitudes. El saldo prueba acceso, no la tarifa de un modelo. Consultar precio actual en la página/panel del modelo y registrar fecha, URL y conversión

efectiva del plan.

09. Aprobar presupuesto ad1 antes de enviar tareas

Preparar presupuestos/ad01.md: clips descargados, faltantes, modelos, cantidad, duración/resolución, tarifa actual, créditos, USD, voz, stock/música y reserva. No generar para «probar cuánto cuesta» sin presupuesto aprobado. Mantener un ledger de consumo y tareas pendientes; el tope incluye lo enviado aunque aún no termine. Tras autorización explícita de la persona, Claude registra presupuestos/aprobacion-ad01.json. Este ejemplo empieza deliberadamente en false; el agente no lo convierte en true hasta recibir la autorización correspondiente:

```
{
  "approved": false,
  "approved_by": "persona",
  "approved_at": "FECHA_Y_HORA_REAL",
  "authorization_text": "MENSAJE_REAL_DE_AUTORIZACION",
  "scope": "ad01",
  "max_credits": 200,
  "max_usd": 1.00,
  "allowed_models": [
    "gpt-image-2-text-to-image",
    "gpt-image-2-image-to-image",
    "bytedance/v1-pro-fast-image-to-video",
    "elevenlabs/text-to-speech-multilingual-v2",
    "kling/ai-avatar-standard"
  ]
}
```

200 créditos / US$1.00 son un ejemplo de estructura, no un presupuesto ni tarifa aprobados para otra persona. Completar valores desde el presupuesto real. El script comprueba presencia de aprobación/modelo, pero no puede verificar la autoría del mensaje ni imponer el presupuesto al proveedor: Claude controla la autorización y el gasto acumulado antes de cada envío. No reenviar una tarea por timeout. Conservar registro y consultar el historial Kie para saber si ya fue creada/cobrada. Un nuevo POST puede generar un segundo cobro. No cambiar a pro ni usar otro modelo fuera del alcance autorizado.

10. API de imagen exacta y descarga

Guardar prompts/C001-imagen.json con un prompt derivado del clip observado:

```
{
  "model": "gpt-image-2-text-to-image",
  "input": {
    "prompt": "Raw potato in hand, macro vertical, warm kitchen light, natural texture, no text.",
    "aspect_ratio": "9:16",
    "resolution": "1K"
  }
}
```

Claude redacta el prompt según la papa/acción real de Watch; este ejemplo muestra el schema, no obliga a inventar una papa en todos los anuncios. Los prompts JSON son ejemplos breves: Claude los amplía con todos los atributos observados por Watch antes de enviar. Usar resolution, no copiar image_size/output_format de memorias antiguas sin comprobar el schema actual. Crear tarea, consultar y descargar mediante el script incluido:

```
"$ADS_PYTHON" "$ADS_SKILL/scripts/kie.py" submit \
  prompts/C001-imagen.json \
  --approval presupuestos/aprobacion-ad01.json \
  --record logs/kie/C001-imagen.json
"$ADS_PYTHON" "$ADS_SKILL/scripts/kie.py" poll \
  logs/kie/C001-imagen.json --wait 50
```

```
"$ADS_PYTHON" "$ADS_SKILL/scripts/kie.py" download \
  logs/kie/C001-imagen.json 04-assets/generados/imagenes/C001.png
```

submit realiza POST https://api.kie.ai/api/v1/jobs/createTask con JSON y Bearer; guarda data.taskId. poll realiza GET /api/v1/jobs/recordInfo?taskId=<taskId>. Si sigue waiting/queuing/generating, conservar el ID y repetir poll, no submit. En success interpreta resultJson tanto si es string JSON como si es objeto, toma resultUrls y descarga archivo. En fail registra failCode/failMsg. El archivo CDN se descarga sin enviarle la clave de Kie. Para imagen con referencias usar model=gpt-image-2-image-to-image e input.input_urls=[URL_PUBLICA_DE_REFERENCIA] junto con prompt/aspect_ratio/resolution. Inspeccionar el resultado antes de animar; si falla objeto/manos/texto, corregir prompt y presupuestar el reintento. No aceptar una foto incorrecta para después intentar arreglarla con motion.

11. Subir archivos a Kie y generar b-roll en video

Un modelo no lee /Users/... ni una ruta local de Windows. Necesita una URL accesible. Para imagen descargada o avatar propio, el cliente hace multipart/form-data:

```
"$ADS_PYTHON" "$ADS_SKILL/scripts/kie.py" upload \
  04-assets/generados/imagenes/C001.png \
  --record logs/kie/C001-upload.json
```

Endpoint de upload: POST https://kieai.redpandaai.co/api/file-stream-upload. Campos: file (binario), uploadPath=ads/sop y fileName único. Requiere Bearer; leer success=true, code=200 y data.downloadUrl. Los uploads son temporales según el proveedor; guardar originales localmente y no confiar en esa URL como archivo permanente. Puede usarse resultUrls de una imagen recién generada si sigue accesible; para reanudar después, subir el archivo local. Guardar prompts/C001-video.json copiando la URL real devuelta, nunca una URL de ejemplo:

```
{
  "model": "bytedance/v1-pro-fast-image-to-video",
  "input": {
    "prompt": "Hand rotates same potato once. Locked macro. Stable identity, no new objects.",
    "image_url": "URL_REAL_DEL_UPLOAD_O_RESULTADO",
    "resolution": "720p",
    "duration": "5"
  }
}
```

Enviar y consultar con la misma receta de la sección 10, usando logs/kie/C001-video.json; descargar a 04-assets/generados/video/C001.mp4. duration es string "5" o "10" en este modelo. El video toma encuadre de la imagen; no añadir aspect_ratio como si todos los modelos aceptaran los mismos campos. Revisar inicio/medio/final y reproducción completa: giro real, objeto estable, manos coherentes. Elegir el tramo que corresponda a la duración de Watch. Generar cinco segundos no obliga a poner los cinco en el anuncio. No resolver una acción con una foto movida. Evitar ruido de audio nativo en b-roll cuando el anuncio usa voz separada. Si esta ruta no alcanza la calidad de una escena compleja, comparar una alternativa del catálogo oficial, leer su schema y presentar diferencia de costo antes de exceder el máximo. Veo tiene endpoints propios y no se llama con este payload genérico. No mezclar campos de modelos diferentes.

12. Voz exacta con ElevenLabs vía Kie

Guardar guion completo localizado en 00-brief/guion-es.txt. No resumirlo para que entre en clips más cortos. Elegir voz del catálogo actual de este modelo y escuchar su preview en https://static.aiquickdraw.com/elevenlabs/voice/<voice_id>.mp3. Elegir según idioma, acento, tono y referencia; el ID del ejemplo es James, no una certificación de acento rioplatense. Guardar prompts/ad01-voz.json:

```
{
  "model": "elevenlabs/text-to-speech-multilingual-v2",
```

```
  "input": {
    "text": "GUION_COMPLETO_LOCALIZADO",
    "voice": "EkK5I93UQWFDigLMpZcX",
    "stability": 0.5,
    "similarity_boost": 0.75,
    "style": 0.2,
    "speed": 1.05,
    "timestamps": true
  }
}
```

Los valores de actuación son punto inicial ajustable a la referencia; no acelerar todas las ofertas igual. Crear tarea/poll/download como en 10, con logs/kie/ad01-voz.json y salida 05-audio/voz/voz-es.mp3. Máximo de texto según schema: 5000 caracteres. Si lo excede, dividir por frases/bloques, mantener misma voz y usar previous_text/next_text para continuidad; conservar concatenación y tiempos finales. No confiar en que timestamps:true devuelve automáticamente un SRT listo. Inspeccionar la respuesta y convertir el formato real a captions; si no entrega tiempos útiles, transcribir la voz descargada con Whisper y corregir con el guion original. Revisar pronunciación de marca, cifras, idioma y CTA escuchando el archivo. Clonación de voz solo autorizada; no clonar automáticamente al actor de la referencia. Para réplica de una referencia con voz original del usuario ya aprobada, puede reutilizarse ese audio si corresponde al idioma/guion; no gastar TTS nuevamente por rutina. Cualquier alternativa gratuita se declara y valida antes de sustituir el acabado.

13. UGC propio: retrato + voz + Kling

Si Watch muestra una persona hablando y el formato lo requiere, reconstruir actuación, plano, escenario y continuidad con avatar propio. Generar retrato con gpt-image-2 en 1K, inspeccionarlo y subirlo. Producir voz completa de ese tramo con ElevenLabs y subirla. Kie necesita URLs de ambos archivos; no necesita que la persona de la referencia sea clonada. Guardar prompts/ad01-ugc.json:

```
{
  "model": "kling/ai-avatar-standard",
  "input": {
    "image_url": "URL_REAL_RETRATO",
    "audio_url": "URL_REAL_VOZ",
    "prompt": "Medium close-up, natural gestures, warm expression, stable camera and identity."
  }
}
```

Enviar con el mismo createTask/poll/download y guardar en 04-assets/ugc/ad01-ugc.mp4. prompt es requerido y no vacío. Foto JPEG/PNG y audio accesibles; verificar límites actuales antes de subir. El speech viene de audio_url; no meter guion distinto en prompt. Si labios/actuación no pasan QC, evaluar kling/ai-avatar-pro con presupuesto y schema confirmados; no reenviar sin revisar costo. En split-screen colocar UGC y b-roll como capas de HyperFrames según Watch. En sketch preservar todos los personajes/turnos; en cartoon usar imágenes/animación correspondientes. No convertir todos los formatos en un avatar narrando ni insertar UGC en referencias que no lo tienen.

14. Música y SFX: fuente, archivo y licencia

Primero consultar catálogo propio por ritmo, emoción y duración. Después abrir https://elements.envato.com/ en Chrome conectado, buscar «Music» con descripción musical de Watch (ej.: playful percussion light upbeat, sin voz), escuchar candidatos y descargar el elegido con licencia/proyecto registrado. Guardar cama.mp3 o WAV en 05-audio/musica y evidencia de licencia en licencia.txt. No descargar previews con marca de agua como track final ni declarar sesión/licencia Envato si falta acceso. Alternativa concreta: https://incompetech.com/music/royalty-free/music.html; elegir canción que sirva, leer su licencia específica y guardar atribución cuando la exija. No copiar música de TikTok presumiendo que queda habilitada para anuncios. Si la canción original se identifica y está autorizada, puede usarse; si no, elegir equivalente por pulso/tono/estructura y declarar diferencia. SFX: catálogo propio o Envato, archivos individuales en 05-audio/sfx (ej.: pop.wav, page-flip.wav, click.wav). Cada efecto se coloca en la acción/entrada correspondiente. No reproducir un whoosh cada palabra. Escuchar mezcla completa y registrar si el sonido es propio, stock o generado. No usar Suno automáticamente si ya existe música adecuada; generación adicional requiere presupuesto. Preparar WAV cuando haga falta:

```
ffmpeg -i 05-audio/voz/voz-es.mp3 -ar 48000 -ac 2 \
  05-audio/voz/voz-es.wav
```

La mezcla de voz/música/SFX colocados se hace con la capacidad audio de HyperFrames. Ganancias iniciales son ajustables al archivo; verificar inteligibilidad escuchando y picos con ffmpeg, no fijar el mismo volumen por porcentaje para cualquier track.

15. HyperFrames: crear proyecto y componer la línea de tiempo

Leer hyperframes, hyperframes-core y CLI; cargar animation/keyframes/audio/registry/media-use según lo que se va a hacer. Instalar skills con npx hyperframes skills una vez; comprobar lo instalado. Antes de producción, consultar uso de Claude si disponible:

```
npx hyperframes usage --harness claude-code --json
npx hyperframes init "$ADS_PROJECT/06-hyperframes/ad01" \
  --resolution portrait --non-interactive
cd "$ADS_PROJECT/06-hyperframes/ad01"
npx hyperframes catalog --query "kinetic word highlight"
```

Claude construye index.html y proyecto scaffolded conforme al contrato de HyperFrames instalado: clips con data-start/data-duration/data-media-start, tracks, runtime seekable y media administrada por framework. Antes de escribir animaciones manuales, buscar componente pertinente en catalog; no inventar imports ni copiar una composición incompatible con la versión instalada. Mapear C001 ... Cnn a timeline. Voz final define sincronía: repetir argumento completo; ajustar duración de planos a la traducción y mantener secuencia/formatos observados. Capas específicas para video, captions, overlays y audio. Colocar entrada/permanencia/salida de cada gráfico y no cortar una acción antes de que se entienda. Guardar storyboard/timeline y prompts. Dinamismo continuo: cada bloque lleva estímulo específico ligado a lo que se dice. Alternar acciones, demostraciones, persona/producto, corte, énfasis de palabra, popup y SFX sincronizado. No sostener durante todo un argumento una foto sin evolución. La cadencia deriva de acciones y referencia; no regla ciega de dos segundos. Ráfagas cortas sirven donde la acción lo permite, pausas más largas donde hay que entender una prueba. Materiales: mostrar páginas propias nítidas y animadas, fondo relacionado desenfocado cuando aporte, relevos laterales/zoom suave y demostración concreta. Popups aportan dato/beneficio, no adornos genéricos. No tapar producto con subtítulos ni bandas gigantes. Mantener guion completo y CTA propio. Al reeditar VSL con narradora existente, preservar encuadre original sin crop/zoom para ocultar textos y evitar subtítulos duplicados. En edición de un proyecto existente, consultar timeline antes de tocar tiempos:

```
npx hyperframes timeline --json
npx hyperframes lint
npx hyperframes check
npx hyperframes snapshot --at 1,10,20
```

```
npx hyperframes preview --background
```

check debe pasar o documentar/reparar fallos. Revisar preview completo; una captura del hook no acredita el resto. Cuando el usuario ya autorizó producir ad1 completo dentro del presupuesto, realizar su render para entregarlo a revisión; no insertar una aprobación redundante solo por este manual. Si pide detenerse en preview, respetarlo.

16. Render, comparación literal y QC

Después de componer y revisar:

```
npx hyperframes render --quality delivery \
  --output "$ADS_PROJECT/07-renders/ad01-v01.mp4"
ffprobe -v error -show_format -show_streams \
  "$ADS_PROJECT/07-renders/ad01-v01.mp4"
ffmpeg -i "$ADS_PROJECT/07-renders/ad01-v01.mp4" \
  -af volumedetect -f null -
ffmpeg -i "$ADS_PROJECT/07-renders/ad01-v01.mp4" \
  -vf "blackdetect=d=0.1:pix_th=0.10,freezedetect=n=-50dB:d=1" \
  -an -f null -
```

Verificar formato vertical esperado, resolución real, fps y duración; los checks automáticos ayudan pero requieren juicio: una pausa visual deliberada no es necesariamente fallo. Ver el MP4 entero con audio, comprobar sincronía de voz, captions, música, SFX, lip-sync y CTA. Si no puede escucharse, declarar pendiente y no afirmar QC audiovisual. Completar 08-qc/correspondencias.csv por cada Cnn: tiempo final, asset utilizado, coincidencia de objeto/acción/formato, diferencias y motivo. Exigir papa cuando la referencia muestra papa, mismo gesto cuando ese gesto es el hook, split-screen donde lo hay. Fidelidad no significa copiar claims sin respaldo ni marca ajena. No declarar réplica exacta si hay diferencias. Mostrar original/resultado con archivos reproducibles en 09-entrega/preview.html, costo estimado/real y proyecto editable. Ad1 queda en revisión; incorporar feedback en brief/receta, no producir los otros cinco mientras no se apruebe.

17. Ads 2-6, aprendizaje y cierre

Con ad1 aprobado, inventariar las siguientes cinco referencias en el orden de biblioteca y repetir Watch completo, sourcing literal y mapa de correspondencias. Presupuestar cada necesidad y reuso, no multiplicar el costo de ad1 por cinco. Guardar presupuestos/ads02-06.md y su aprobación propia; usar alcance y ledger separados. La tanda son seis, ad1 incluido. Producir y revisar las cinco, conservar su formato y argumentos. Preparar copy Meta por anuncio, preview de las seis y proyecto editable por pieza en 06-hyperframes/ad02 ... ad06. Modeler/modelar actualiza su panel existente. Publicar sigue en la fase de lanzamiento y requiere la orden correspondiente; este instalador no crea campañas. Registrar correcciones aprobadas por formato. Catalogar assets con objeto, acción, encuadre, autor, URL, derechos, SHA-256, ruta, modelo/task/consumo y usos. Multimedia activa se conserva mientras se revisa. Al cerrar, copiar a almacenamiento remoto configurado, comprobar contenido y guardar manifiesto antes de retirar local. En este proyecto el destino es Cloudflare R2 mediante su herramienta de archivado; para otra persona se configura su propio remoto. No ejecutar borrado sin copia verificada.

18. Diagnóstico sin gasto y prompt de instalación

Adjuntar PDF y ZIP a Claude y pegar. Si se dispone solo del PDF, Claude reconstruye los archivos copiando los scripts completos del anexo, con los nombres indicados, y después ejecuta la misma instalación:

```
Instala este SOP exacto v2 para Claude en mi fase de anuncios de modeler/modelar.
Lee todo el PDF y el paquete. Ejecuta instalar.py primero con --dry-run.
Respalda archivos y preserva mis fases de oferta, tienda y lanzamiento.
Configura Google Chrome + Claude in Chrome; comprueba tools y conexión.
Usa TikTok/Pinterest en ese Chrome y SSSTik para la descarga TikTok.
```

```
Crea las carpetas con iniciar-proyecto.py y conserva CSV, prompts y logs.
Configura KIE_API_KEY sin imprimirla y comprueba saldo con kie.py credits.
Usa los slugs y payloads de este manual, no DALL-E ni Nano Banana genéricos.
Instala HyperFrames y las dependencias; registra versiones.
No hagas generaciones pagas ni campañas durante la instalación.
Al terminar muestra rutas, archivos respaldados, tools disponibles y pendientes.
Después analiza una referencia completa y prepara sourcing/presupuesto ad1.
Espera mi aprobación del costo antes de producir ad1 y mi revisión antes de cinco más.
```

Prueba de aceptación: 1) skills descubiertas y links existentes; 2) Chrome abre pestaña propia y busca una consulta literal; 3) descarga autorizada de prueba y ffprobe valida el archivo en carpeta; 4) saldo Kie accesible sin generar; 5) JSON de modelos válido; 6) HyperFrames init/check de prueba sin generación IA; 7) cliente bloquea submit con approved=false; 8) Watch cubre todos los clips. Una herramienta no disponible queda pendiente, no instalada por declaración. Los scripts incluidos se verifican con pruebas locales simuladas, sin consumir créditos. Los endpoints/schemas se consultaron en documentación oficial el 2026-10-10. La sesión de Chrome/SSSTik de cada persona y una llamada paga real no se certifican mediante este documento; se validan en instalación/producción.

19. Fuentes y alcance de exactitud

Operación documentada: WIKI/produccion-creativos-propios.md, WIKI/kie-ai-generacion.md, WIKI/landing-ceramic-molds-catalog.md (SSSTik), memorias de imagen GPT y edición dinámica, y OUTPUTS/2026-10-10-carton-circuitos-prueba/costos-reales.json (GPT Image 2 + ByteDance Fast). El árbol de carpetas, scripts portables y umbral de búsqueda son estandarización operativa nueva de este manual; no se presentan como copia literal de un procedimiento previo que no existía documentado. Documentación oficial para comprobación futura:

- Claude + Chrome: https://code.claude.com/docs/en/chrome

- Instalar Claude: https://code.claude.com/docs/en/setup

- Skills Claude: https://code.claude.com/docs/en/skills

- SSSTik: https://ssstik.io/es

- GPT imagen: https://docs.kie.ai/market/gpt/gpt-image-2-text-to-image

- GPT edición: https://docs.kie.ai/market/gpt/gpt-image-2-image-to-image

- ByteDance: https://docs.kie.ai/market/bytedance/v1-pro-fast-image-to-video

- Voz: https://docs.kie.ai/market/elevenlabs/text-to-speech-multilingual-v2

- UGC: https://docs.kie.ai/market/kling/ai-avatar-standard

- Upload: https://docs.kie.ai/file-upload-api/upload-file-stream

- Saldo: https://docs.kie.ai/common-api/get-account-credits

- HyperFrames: https://hyperframes.heygen.com/developers

Si cambia una interfaz o un schema, Claude inspecciona el estado/documentación actual, anota el ajuste y conserva el objetivo y prioridades; no simula un clic o campo obsoleto. Ningún manual puede fijar para siempre botones de terceros ni garantizar idéntico resultado de una generación estocástica.

SOP de anuncios: reconstrucción fiel y edición dinámica

Versión 2026-10-10. Contrato portable para modeler/modelar, Watch y ad-creator.

1. Resultado y responsabilidades

Modeler/modelar orquesta. Watch observa y documenta todos los clips y formatos, con un mapa que permita replicarlos tal cual. Ad-creator busca, descarga, genera faltantes, consigue música, hace voz y compone en HyperFrames. Watch no genera anuncios: su análisis debe ser suficientemente preciso para reconstruirlos. La referencia determina el formato, las acciones, los objetos y el orden. Se busca máxima fidelidad visual con recursos disponibles o generados, localizando idioma, marca, oferta y producto propios. No reemplazar una acción específica por un clip genérico del mismo nicho. Si el clip 1 muestra una papa, reconstruir una papa, su acción, encuadre, contexto y duración. No basta una cocina o una verdura distinta.

2. Entrada y preparación

Recibir referencia/arsenal o archivo, oferta/landing, mercado, idioma/acento y estado del proyecto. Recuperar información existente; preguntar solo por datos necesarios que falten. Mantener el orden visible del arsenal, registrar referencias repetidas y no inventar ranking ni cifras. Si ya existe ad1 aprobado, reanudar desde ese estado. Preparar carpetas referencias/, analisis/, recursos/, audio/, hyperframes/, exportaciones/ y registros/. Nunca poner credenciales en documentos o código. Mantener secretos en variables de entorno o gestor seguro.

3. Watch completo: cada clip y cada formato

Ver el video entero y escuchar su audio. Detectar cambios de plano y revisar cada segmento en movimiento, incluidos clips muy cortos, insertos, pantallas divididas, overlays y cambios de formato dentro del mismo anuncio. Los frames apoyan la revisión; no sustituyen observar acciones ni escuchar audio. Si falta acceso, marcar exactamente qué tramo/dimensión no pudo analizarse; no declarar análisis completo. Transcribir el discurso completo con timestamps. Separar voz, textos en pantalla y copy externo. Conservar orden de argumentos y guion íntegro al localizar, sin resumir ni reescribir salvo instrucción explícita. Adaptar claims ajenos únicamente a lo que la oferta propia pueda respaldar. Por cada clip registrar: ID, inicio/fin, duración, frase, formato y capas; objeto/persona literal y atributos; acción y sus fases; escenario, encuadre, cámara, iluminación y estilo; subtítulos, popups, movimientos y transiciones; música/SFX y sus momentos; función narrativa. Describir UGC por actuación, gestos y continuidad. No inventar identidad, canción o rendimiento. Entregar transcripción, mapa exhaustivo de clips, inventario de formatos y recursos, y criterios de fidelidad por clip. Añadir una columna de elementos imprescindibles y otra de diferencias permitidas. La producción convertirá este mapa en búsquedas y prompts; Watch no necesita producirlos.

4. Sourcing: buscar antes de generar

Para cada b-roll, primero buscar en internet, TikTok y Pinterest. Usar consultas por objeto + acción + encuadre, en idioma original, destino y variantes útiles. Ejemplo: papa pelada girando en mano, primer plano vertical; potato peeling close up vertical. Revisar candidatos en movimiento y descargar los útiles mediante métodos disponibles y autorizados, conservando autor, URL, tramo, licencia/permisos y ruta local. Registrar búsquedas realizadas, candidatos, descartes y fallos de acceso. Si una plataforma no es accesible, indicar ese límite y seguir por fuentes accesibles; no afirmar que se buscó allí. Pinterest puede enlazar al creador: recuperar la fuente y el video original cuando sea posible. Tras esa primera búsqueda, consultar biblioteca propia y stock/Envato para completar faltantes. No imponer un recurso existente que no coincida. Descargar solo lo necesario; registrar restricciones. Material públicamente visible no equivale a licencia publicitaria. No quitar marcas para eludir derechos; Vmake no es requisito ni base del proceso. Elegir por fidelidad de objeto, acción, encuadre y formato, además de calidad técnica. Una escena queda resuelta solo con un archivo real revisado, o una generación necesaria planificada; un enlace o thumbnail no es un recurso descargado.

5. Generar únicamente lo necesario

Generar con Kie los clips que no se encontraron adecuados y los elementos necesariamente propios: UGC, avatar, demostración del producto, personaje o continuidad específica. Un UGC necesario puede planificarse directamente sin buscar una persona idéntica ni clonar identidad/voz ajena. Mantener objeto y acción de la referencia, identidad del personaje, vestuario, luz y estilo entre escenas. El prompt deriva del clip observado: objeto + acción + entorno + encuadre + cámara + duración + estilo + restricciones. Para la papa, generar la papa concreta en la acción descrita, no un montaje culinario genérico. Revisar manos, objeto, continuidad, texto quemado y movimiento. Una imagen con zoom no sustituye un clip de acción; si la referencia usa imágenes, sí respetar ese formato. Comparar modelos/resoluciones actuales y elegir el menor costo que pase calidad. No fijar modelos, tarifas o conversiones históricas. Registrar prompts, parámetros, task ID, resultado, fallos, consumo y motivo de regeneración.

6. Presupuesto y secuencia de seis anuncios

Preparar sourcing, storyboard y presupuesto sin generar gastos. Antes de cualquier tarea paga del ad1, mostrar créditos y USD por recurso: voz, música/stock, imágenes, video, UGC/lipsync, reserva y máximo. Verificar tarifa y conversión actual en fuente oficial/panel. Esperar aprobación explícita del presupuesto; respetar una autorización ya dada para ese tramo y tope. Producir solo ad1 completo y mostrarlo para revisión con costo real. Incorporar correcciones al proceso. Con ad1 aprobado, presupuestar los cinco adicionales por necesidades reales y reutilización; esperar aprobación explícita del gasto incremental antes de producir ads 2-6. La tanda normal son seis totales, ad1 incluido. Una prueba de cantidad explícita respeta esa cantidad. No repetir permisos ya otorgados dentro de su alcance. Si el costo va a superar el máximo autorizado, parar ese gasto y presentar el ajuste. Aprobar creatividad no autoriza publicar campañas ni demuestra ventas.

7. Música, voz y sonido

Conseguir música adecuada al ritmo, carácter y estructura observados; usar la misma solo si su uso está autorizado, o una alternativa equivalente licenciada. Descargar el archivo, guardar procedencia/licencia y ajustar entradas, cortes y cierre. No inventar el título si Watch no lo identificó. Generar voz localizada con proveedor TTS configurado, por ejemplo ElevenLabs o Gemini TTS si están disponibles; verificar idiomas, tarifas y condiciones actuales. Mantener discurso completo, tono, velocidad, emoción y pausas coherentes con la referencia. No clonar voz sin autorización. Preparar pronunciación de marca, cifras y CTA; revisar el audio antes de componer. Si hay UGC, usar esa voz para lipsync y comprobar sincronía. Conseguir SFX pertinentes para acciones, popups, transiciones y énfasis. Ajustar música debajo de voz y evitar clipping, silencios accidentales, repetición mecánica o efectos que tapen el mensaje. Registrar archivos y tiempos.

8. Composición dinámica en HyperFrames

Leer las skills y el contrato vigente de HyperFrames antes de componer: core, CLI y dominios de animación, keyframes, audio, registry y media-use según necesidad. Componer allí clips, formatos, voz, música, SFX, captions y overlays. FFmpeg puede preparar recursos y verificar exportaciones; la composición final pertenece a HyperFrames. Reconstruir el orden, formatos, acciones y ritmo de la referencia. Si cambia de UGC a split-screen y luego a demostración, conservar esos cambios. Ajustar tiempos a la voz localizada sin cortar argumentos. No imponer una plantilla única a los seis anuncios. Mantener estímulos durante todo el anuncio: acción visible, cambios de plano motivados, movimiento, énfasis de captions, popups semánticos y sonido sincronizado. Planificar cada bloque con un estímulo y una razón ligada al discurso. El dinamismo no consiste en repetir el mismo zoom o whoosh ni llenar cada palabra de efectos. Alternar tensión, prueba, alivio, beneficios, oferta y CTA; conservar legibilidad y producto visible. La cadencia sigue acciones y referencia; no es obligatorio cortar cada dos segundos. Auditar tramos largos sin evolución y resolverlos con acción, inserto, gráfico o variación sonora pertinentes. No alterar encuadre de narradora al reeditar VSL existente: plano completo sin crop/zoom para ocultar textos, sin bandas grandes ni subtítulos duplicados. Subtítulos compactos y sincronizados, énfasis en palabras relevantes, gráficos con entrada/permanencia/salida y zonas seguras de móvil. Usar material propio real para páginas, pack, beneficios y CTA. Evitar que textos tapen lo que se demuestra.

9. Control de calidad y entrega

Comparar original y resultado clip por clip. Cada ID de Watch debe mapear a un tramo final; comprobar objeto, acción, formato, encuadre, orden y ritmo. Registrar diferencias y motivo; no afirmar réplica exacta si hay desvíos. Buscar discrepancias literales como papa reemplazada por otra cosa, acción perdida o formato cambiado. Ver el render entero con audio: pronunciación, captions, continuidad, lipsync, música/voz, SFX, negros, congelados, recortes, legibilidad, CTA y duración. Revisar hook, cuerpo y cierre también con snapshots. Si no pudo escucharse, reportar QC de audio pendiente. Exportar por defecto MP4 vertical 1080x1920, verificando compatibilidad del destino y propiedades reales. Entregar ad1 reproducible para revisión, después preview de ad1-ad6 con copy y costos, mapa original/resultado, manifiesto de recursos y proyecto editable. Modeler enlaza todo al panel existente; el lanzamiento mantiene su fase y autorización. Guardar feedback por formato y las correcciones aprobadas, sin convertir preferencias en performance. Catalogar recursos por objeto, acción, encuadre, estilo y derechos. Archivar multimedia al cerrar con copia y verificación antes de retirar local; recuperar recursos del catálogo en trabajos futuros. El almacenamiento concreto lo configura cada persona.

10. Evidencia mínima de una ejecución válida

analisis/watch.md y clips.csv: todos los segmentos y formatos, con observaciones/limitaciones. registros/busquedas.csv: clip, plataforma, consulta, candidato, selección/descarte y acceso. recursos/manifest.csv: clip, archivo, fuente/URL, derechos, tramo, QC y generación si aplica. registros/presupuesto.md y costos.csv: estimación, máximo, aprobaciones, consumo y fallos. audio/: voz, música y SFX reales. hyperframes/: composición editable y parámetros. registros/qc.md: comparación por clip, revisión audiovisual y diferencias. exportaciones/: MP4 y preview.

11. Prueba de aceptación del SOP instalado

Pedir un análisis de una referencia corta con tres o más clips y un cambio de formato. Watch debe cubrir todos los clips, incluyendo el objeto literal, sin generar assets. Ad-creator debe producir un plan de búsqueda internet/TikTok/Pinterest y mapa de recursos antes de proponer IA para b-roll faltante. Un UGC propio necesario puede ir directamente a generación planificada. Pedir presupuesto sin gastar. Verificar que el flujo se detenga antes de una tarea paga sin aprobación y que espere revisión de ad1 antes de los otros cinco. Tras autorización, verificar música/voz reales y composición HyperFrames con estímulos de principio a fin. Si falta alguna herramienta, informar qué falta sin simular éxito.

VSL existente: edición dinámica con encuadre original

Aprobación y alcance

Gerónimo aprobó la v3 de «Paz en tu Hogar» el 2026-10-07: «quedó muy perfecto, aprende eso en el SOP». Referencia aprobada: `vsl-paz-en-tu-hogar-v3-encuadre-original.mp4`, documentada en `OUTPUTS/2026-10-07-vsl-antiguo-paz-en-tu-hogar-reeditado.md`. Es aprobación del acabado creativo de esta reedición; no hay métricas comerciales.

Aplicar al reeditar una VSL existente con narradora/presentador. Conservar el encuadre completo del original, salvo pedido explícito de cambiarlo. No extender a otros formatos decisiones particulares como esta paleta, duración o cantidad de cortes.

Montaje aprobado

1. Usar el video original completo, centrado y sin zoom ni crop sobre la narradora. Ajustar proporcionalmente al lienzo (`object-fit: contain` en HyperFrames); exportar a otra resolución no autoriza recortar para llenar el frame.

2. Conservar voz íntegra y orden de argumentos. En esta referencia fueron 4:20; no fijar esa duración para futuras VSL ni alargar o resumir un original para imitarla.

3. Dar dinamismo con alternancia de narradora, b-roll y gráficos relacionados con cada frase: conflicto cotidiano, distancia, repetición, comprensión, solución, contenido, garantía y CTA. Las placas de mecanismo/oferta son breves —aproximadamente 2–3 segundos en esta pieza— y vuelven al rostro o a una escena real. No imponer una frecuencia de corte universal.

4. Usar la imagen completa disponible. No añadir bandas bordó, footers ni recuadros permanentes que consuman una parte grande de la pantalla. Los textos y tarjetas no deben tapar rostro ni gestos relevantes.

5. Conservar los subtítulos incrustados cuando se ve el original, si ya son legibles. Mostrar los nuevos solo cuando b-roll o una gráfica reemplazan la imagen. En esta receta, frases de hasta tres palabras, énfasis selectivo y respaldo compacto; nunca dos capas de subtítulos simultáneas. No recortar ni ampliar el plano para esconder textos antiguos. Si hace falta limpiar el original, resolverlo sin destruir el encuadre y bajo el presupuesto autorizado.

6. En stock horizontal, elegir un encuadre que conserve la acción y los interlocutores. En la discusión de pareja funcionaron dos recortes complementarios; no hacer un crop central que pierda ambas caras ni usar una banda enorme para rellenar el espacio.

7. Música acorde al paso tensión → alivio, voz al frente, carve/ducking y SFX puntuales en acciones o argumentos. No llenar cada palabra de efectos ni volver a generar el montaje borrando los ajustes de audio.

8. Producto, calendario, mapa y tarjetas ilustran lo mencionado; no inventar capturas del producto, pruebas clínicas ni testimonios. Actores de stock no son clientes.

Control antes de entregar

- Comparar original y editado en varios momentos de narradora: mismos bordes, cabeza, manos, cuerpo y gestos visibles. Un render técnicamente válido puede tener un mal crop: revisar el encuadre visualmente.

- Comprobar una sola capa de subtítulos por imagen, legibilidad en móvil y ausencia de solapamiento con tarjetas, CTA o flechas, también durante sus animaciones.

- Revisar hook, escenas de dolor, mecanismo, oferta, garantía y cierre en el MP4 exportado, además del preview. Comprobar duración, audio, sincronía y decodificación; declarar cualquier limitación real de escucha.

- Guardar brief, montaje editable, fuente, derechos, QA y feedback. Entregar revisiones separadas para comparar. Usar como referencia v3 aprobada; v1 con banda grande y v2 con narradora recortada están superadas.

Recuperación

- Proyecto y MP4 activos: `/Users/geronimolahoz/Renovaciones-local/2026-10-07-vsl-40dias-reedit/`.

- Código/guion/QA: `OUTPUTS/2026-10-07-vsl-40dias-reedit/` en Segundo Cerebro.

- Media respaldada y verificada: `r2:segundo-cerebro/OUTPUTS/2026-10-07-vsl-40dias-reedit/`.

- Costo adicional de generación de esta reedición: USD 0. Esto no sustituye presupuestar y aprobar nuevos recursos pagos.

Tres formatos aprobados — 2026-10-07

Gerónimo vio los tres resultados y dijo: «si me encantaron, aprende todo lo que hiciste entonces, gracias». Aprobación de ejecución creativa; todavía no hay métricas de ventas. Estas recetas orientan futuros anuncios según mensaje y referencia, sin congelar una plantilla ni extender autorización de gasto.

Presentador + demostración real (1826257598559732)

- Presentador recortado con voz continua, b-roll de procesos y planos reales de herrería. Alternar explicación, acción, beneficios y oferta; los cortes deben demostrar lo que se dice.

- Look industrial con contraste fuerte, énfasis selectivo en precio/beneficio/CTA y sonido por acción. Reservar espacio para el rostro: en esta pieza, subir los subtítulos evitó taparlo.

- Usar gráficos técnicos ilustrativos identificados como tales; no presentarlos como planos entregables reales.

Sketch con humanos (1816789216183482)

- Traducir íntegros los diálogos y conservar su orden. Humanos fotorealistas en esta adaptación por pedido expreso; no imponer humanos a todas las historias.

- Fijar identidad, vestuario y aspecto mediante imágenes de referencia; generar nuevas escenas desde esas referencias. Mantener continuidad entre casa, gimnasio, oficina y reunión social.

- Lipsync para intervenciones relevantes; reacciones y contraplanos para respuestas cortas y líneas largas. Construir incomodidad, reconocimiento y alivio con actuación y montaje, sin llenar cada palabra de efectos.

- Introducir el producto brevemente cuando el diálogo lo justifica y cerrar con CTA claro.

Historia cartoon animada (1493744079229950)

- Personaje recurrente y su sombra como recurso narrativo; animación real de acciones, no sucesión de imágenes con zoom.

- Estética de novela gráfica adulta, composición por escena y placas breves. En esta prueba funcionaron azul oscuro/bordó/dorado; elegir paleta pertinente en otras ofertas.

- Desarrollar problema → repetición → comprensión → alivio → solución. Sincronizar recursos visuales con situaciones concretas del guion y enfatizar el mecanismo/oferta.

Decisiones técnicas que conviene reutilizar

- Generación económica validada aquí: GPT Image 2 1K para imágenes/continuidad, Grok Imagine 720p para acciones y Kling Avatar Standard para lipsync, vía Kie. Verificar catálogo, tarifas y parámetros actuales antes de cada nuevo presupuesto; esta elección no obliga a usar siempre los mismos modelos.

- Voz continua como reloj del montaje. Dividir audio para límites del avatar, recortar relleno de salida a la duración de cada fragmento y conservar la voz original en el montaje final.

- Timestamps de palabras desde el audio final; corregir errores del ASR contra el guion aprobado. Terminar un subtítulo al iniciar el siguiente, sin solapamiento artificial.

- HyperFrames para montaje gráfico, subtítulos y motion; ffmpeg para preparación de clips, recortes y composición de chroma si hace falta. Guardar el proyecto editable y las rutas de assets.

- Música propia distinta por formato, voz al frente, ducking y SFX ligados a acciones/argumentos. No regenerar el montaje después del ajuste de audio sin reaplicar ese ajuste.

- Revisar el render exportado además del preview: recortes, textos ajenos incrustados, rostro/subtítulos, contraste, continuidad, CTA y movimiento. En SVG de líneas, declarar fill:none para evitar rellenos inesperados.

- Medir picos del audio estéreo sin convertirlo a mono para ese diagnóstico; comprobar sincronía con la voz fuente y buscar negros/congelados. Los controles técnicos no sustituyen escucha y revisión de lipsync: declarar cualquier limitación de inspección.

- TikTok: conservar autor, URL y metadatos; revisar que el crop retire textos originales sin destruir la acción. Esta toma real no tiene permiso comercial verificado y se usó en prueba interna. Envato fue buscado pero no hubo descarga licenciada; no afirmar lo contrario.

Recuperación y costos históricos

- Evidencia, scripts, guiones, QC y proyectos: `/Users/geronimolahoz/Desktop/Segundo Cerebro/OUTPUTS/2026-10-07-tres-formatos/`.

- Media y renders: `/Users/geronimolahoz/Renovaciones-local/2026-10-07-tres-formatos/`; respaldo verificado `r2:segundo-cerebro/_renders/2026-10-07-tres-formatos/`.

- Buscar recursos por acción/emoción/estética en `assets/catalogo-creativos.jsonl`: 74 recursos ingresados, incluyendo buenas tomas no utilizadas. Respetar derechos y restricciones por recurso.

- Costo real de esta tanda: 386 + 576 + 396 = 1.358 créditos / US$6,79; máximo aprobado 1.874 / US$9,37; 64 generaciones completadas, sin fallos. Son cifras históricas de esta tanda, no tarifas ni presupuestos futuros.

- Consultar `generar.py`, `montar.py`, `preparar-base.py`, `audio-propio.py` y `qc-final.py` como ejemplos recuperables; adaptar rutas, escenas y presupuesto. No ejecutar generación paga por estar aprendiendo o reutilizando una receta.

VSL existente con narradora — aprobación 2026-10-07

Gerónimo aprobó la v3 de Paz en tu Hogar y pidió aprenderla en el SOP. Consultar [la receta completa](vsl-encuadre-original.md): encuadre original íntegro sin crop/zoom, voz completa, b-roll y animaciones por argumento, placas breves, imagen sin bandas grandes y subtítulos sin duplicados. Supera v1 y v2 de esta VSL; no cambia las recetas de otros formatos.

Edición dinámica de materiales — referencia aprobada

Aprobación estética: usuario, 2026-10-07: «esta onda de edicion quiero que aprendas a hacer, es mega dinamica y esta tremenda». Análisis: `OUTPUTS/2026-10-07-watch-edicion-dinamica-musical.md`. Fuente local y hash: `OUTPUTS/2026-10-07-edicion-dinamica-referencia/metadata.json`, en Segundo Cerebro. Esta receta conserva el aprendizaje; falta validar su ejecución propia en un render. No es autorización de generación paga, reedición de tandas previas ni campaña.

Cuándo aplicarla

Ads de kits, fichas, manuales, clases, ebooks y recursos prácticos que permitan mostrar problema, uso y contenido. Elegir el acabado por mensaje; no convertir todos los formatos en esta misma plantilla. En modelado fiel mantener el discurso íntegro y el orden de argumentos. Tanda vigente: seis totales, ad1 + cinco.

Qué copiar de la lógica

1. **Una acción por verbo importante.** «Recortar» muestra recortar; «plastificar» muestra plastificar; «organizar» muestra organización. No cubrir una lista de beneficios distintos con un clip genérico largo. Mantener planos donde la acción necesita tiempo para entenderse.

2. **Cadencia que cambia.** Referencia: media aproximada 1,99s por segmento candidato, tramos cortos 0,53–0,77s y demostraciones de hasta 3,37s. Como punto de partida de adaptación, cortes de 1–3s, ráfagas de 0,5–0,9s en enumeraciones y aire de 2–3s en demostraciones/empatía. No usar un temporizador idéntico ni cortar por cortar.

3. **Alternancia de escalas y formatos.** Persona/emoción → manos/objeto → aula/uso → pack → ficha interior → usuario → bonos → página. Evitar una sucesión completa de fotos con el mismo zoom o una sucesión uniforme de clips IA del mismo look. Biblioteca/Envato/TikTok con procedencia y generación para faltantes, según SOP vigente.

4. **Hook que identifica al destinatario.** Primer plano ya tiene acción; headline breve y grande, alto contraste y énfasis de una palabra. La referencia usa fucsia/blanco/amarillo y una leve inclinación durante ~2,5s. Adaptar color y texto al producto; retirar el hook al comenzar el relato. No trasladar esa banda a todos los planos.

5. **Subtítulos compactos y vivos.** Frases cortas legibles en móvil, blanco con énfasis amarillo o color de marca; fondo oscuro ajustado al texto. Relevos por sentido y timestamps de voz; una sola capa. Variar anclaje solo cuando el sujeto/objeto lo necesita, sin saltos arbitrarios. Como inicio a1080×1920: 52–64px, una o dos líneas breves; ajustar por idioma y ocupación. No hacer una gran placa inferior permanente.

6. **Producto realmente mostrado.** Pack inicial establece qué se compra; componentes y páginas interiores muestran cómo es. Un mockup no basta para toda la enumeración. Mostrar el recurso que corresponde a cada frase, con interiores fieles y legibles.

7. **Fondo relacionado y jerarquía.** Duplicar el material como fondo cover desenfocado; mantener un primer plano contain, nítido, con sombra suave y margen. La referencia mantiene el producto mayormente en la mitad superior para dejar subtítulo y espacio. Ajustar composición, sin cortar títulos ni detalles importantes.

8. **Movimiento con entrada, permanencia y salida.** Se observan aumentos suaves de escala y relevos laterales con blur antes de asentarse. Para adaptar, probar entrada150–250ms, permanencia con crecimiento total3–7% y salida100–200ms; son rangos propuestos, no mediciones exactas del original. Conservar nitidez

durante la mayor parte del plano. El texto debe seguir legible durante el movimiento.

9. **Stack visual de oferta.** Bonos juntos → portada individual → interior representativo. Dar un cambio visual a cada componente nombrado. Cantidades, páginas, urgencia, prueba social y testimonios solo de la oferta respaldada; no importar claims del video.

10. **CTA contextual.** Mostrar la página propia en scroll o una demostración de destino, con botón claro y frase de acción, cuando la landing y sus claims estén listos. Dejarlo leíble el tiempo suficiente. No copiar testimonios de tercero ni usar la página como una textura ilegible.

Montaje HyperFrames

Leer `/hyperframes` y las capacidades pertinentes antes de producir. Separar footage, producto/fondo y texto/CTA en capas; animar solo lo que corresponda y dejar transparente cada capa superior. Un único reloj de composición, videos con tiempos locales correctos y narrativa completa. Audio principal continuo en la raíz, incluso al cambiar el plano.

Dejar el storyboard por frase con: objetivo, asset, in/out, encuadre, movimiento durante la permanencia, énfasis y sonido propuesto. Elegir GSAP/easing y desplazamiento por el recurso. Cortes directos en tareas/personas; desplazamiento desenfocado para relevos de fichas cuando aporte. Reservar el zoom para material/producto: si se reedita una VSL con narradora original, manda la receta específica de encuadre original aprobado, sin crop/zoom sobre ella.

No copiar el tiempo total de43,8s si la voz localizada dura más. Mantener el discurso y cambiar tomas internas para conservar cadencia. Animar contenido y evidencias, no solo bordes, labels y una tarjeta CTA repetida.

Audio: pendiente de referencia, criterio de adaptación

La referencia no tuvo escucha perceptual disponible; su música, acento y SFX no están identificados. Para una implementación, escuchar primero el original si el entorno lo permite y revisar la pieza con audio. Si se diseñan música/SFX propios: narración inteligible, música secundaria con ducking, efectos breves motivados por relevos concretos. No etiquetar sonidos propuestos como copiados u observados, ni agregar un whoosh en cada corte por default.

Validación antes de declarar el formato aprendido en ejecución

- Revisar el video continuo, además de pares densos en hook, ráfaga de acciones, entrada/permanencia/salida de fichas, stack y CTA. Los snapshots aislados no prueban motion.

- El objeto y la acción deben coincidir con lo que dice la voz. Producto/interiores propios fieles; títulos legibles y sin crop.

- Caption completo, correcto y sincronizado, sin duplicados, palabras cortadas ni invasión del objeto/CTA.

- Desenfoque recupera nitidez pronto; transiciones sin bordes negros, flashes, frames congelados ni movimientos que tapen información.

- Verificar mezcla e inteligibilidad con escucha y medición; declarar cualquier dimensión no revisada.

- Mostrar ad1 acabado, registrar feedback y actualizar esta receta solo con evidencia. Aprobación estética y reproducción técnica no son resultados comerciales.

Anexo: scripts completos para instalar desde este PDF

Guardar los siguientes archivos en una carpeta paquete/. Crear además los SKILL.md del bloque siguiente y las referencias de este manual. Estos scripts no contienen credenciales. El ZIP entrega los mismos archivos listos para usar.

Archivo: instalar.py

```
#!/usr/bin/env python3
"""Instala procedimientos de Claude sin sustituir fases existentes."""
from pathlib import Path
import argparse, datetime, shutil

p = argparse.ArgumentParser()
p.add_argument("--target", default=str(Path.home() / ".claude/skills"))
p.add_argument("--dry-run", action="store_true")
a = p.parse_args()
src = Path(__file__).parent
target = Path(a.target).expanduser()
stamp = datetime.datetime.now().strftime("%Y%m%d-%H%M%S-%f")
backup = target.parent / "respaldos-sop-anuncios" / stamp
roles = {
    "ad-creator": "ad-creator",
    "watch": "watch",
    "modeler": (
        "modeler"
        if (target / "modeler").exists()
        else "modelar" if (target / "modelar").exists() else "modeler"
    ),
}
for source, name in roles.items():
    dest = target / name
    folder = src / source
    for f in folder.rglob("*"):
        if not f.is_file():
            continue
        rel = f.relative_to(folder)
        out = dest / rel
        print(
            ("MERGE" if rel == Path("SKILL.md") and out.exists() else "COPY")
            + " "
            + str(out)
        )
        if a.dry_run:
            continue
        out.parent.mkdir(parents=True, exist_ok=True)
        if out.exists():
            b = backup / name / rel
            b.parent.mkdir(parents=True, exist_ok=True)
            shutil.copy2(out, b)
        if rel == Path("SKILL.md") and out.exists():
            text = out.read_text()
            marker = "\n## SOP Claude exacto 2026-10-10 v2"
            if marker not in text:
                text += (
                    marker
                    + "\nAntes de la fase de creativos leer "
                    + (
                        "references/sop-claude-exacto.md y references/sop-anuncios.md."
                        if source == "ad-creator"
                        else ("../ad-creator/references/sop-claude-exacto.md y "
                        "../ad-creator/references/sop-anuncios.md.")
                    )
                    + (" Aplicar sus pasos de Chrome, SSSTik, carpetas y API. "
                      "En sourcing y fidelidad prevalecen sobre instrucciones históricas. "
                      "No cambiar fases de oferta, tienda ni lanzamiento.\n")
                )
            out.write_text(text)
        else:
            shutil.copy2(f, out)
if not a.dry_run:
```

```
print("RESPALDO=" + str(backup))
```

Archivo: ad-creator/scripts/iniciar-proyecto.py

```
#!/usr/bin/env python3
import argparse, csv, json, re
from pathlib import Path

p = argparse.ArgumentParser()
p.add_argument("slug")
p.add_argument("--base", default=str(Path.home() / "Renovaciones-local"))
a = p.parse_args()
if not re.fullmatch("[a-z0-9-]+", a.slug):
    raise SystemExit("Slug solo minúsculas, números y guiones")
r = Path(a.base).expanduser() / a.slug
for d in [
    "00-brief",
    "01-referencias/originales",
    "01-referencias/frames",
    "01-referencias/audio",
    "02-watch",
    "03-busquedas",
    "04-assets/descargados/tiktok",
    "04-assets/descargados/pinterest",
    "04-assets/stock",
    "04-assets/generados/imagenes",
    "04-assets/generados/video",
    "04-assets/ugc",
    "05-audio/voz",
    "05-audio/musica",
    "05-audio/sfx",
    "06-hyperframes",
    "07-renders",
    "08-qc",
    "09-entrega",
    "logs/kie",
    "prompts",
    "presupuestos",
]:
    (r / d).mkdir(parents=True, exist_ok=True)
files = {
    "02-watch/clips.csv": [
        "clip_id",
        "inicio",
        "fin",
        "frase",
        "formato",
        "objeto_literal",
        "accion",
        "encuadre",
        "capas",
        "movimiento",
        "musica_sfx",
        "imprescindibles",
    ],
    "03-busquedas/busquedas.csv": [
        "clip_id",
        "plataforma",
        "consulta",
        "url",
        "autor",
        "tramo",
        "decision",
        "motivo",
        "acceso",
    ],
    "04-assets/manifest.csv": [
```

```
        "asset_id",
        "clip_id",
        "archivo",
        "fuente",
        "url",
        "autor",
        "derechos",
        "sha256",
        "duracion",
        "width",
        "height",
        "fps",
        "modelo",
        "task_id",
        "creditos",
        "qc",
    ],
    "08-qc/correspondencias.csv": [
        "clip_id",
        "inicio_final",
        "fin_final",
        "asset_id",
        "objeto",
        "accion",
        "formato",
        "diferencias",
        "motivo",
    ],
}
for f, fields in files.items():
    dest = r / f
    if not dest.exists():
        with dest.open("w", newline="") as stream:
            csv.writer(stream).writerow(fields)
print(r)
```

Archivo: ad-creator/scripts/kie.py

```
#!/usr/bin/env python3
"""Cliente Kie: no imprime secretos ni reenvía POST de generación automáticamente."""
import argparse, json, os, time, uuid
from pathlib import Path
import requests
from dotenv import load_dotenv

API = "https://api.kie.ai"
UPLOAD = "https://kieai.redpandaai.co"
MODELS = {
    "gpt-image-2-text-to-image",
    "gpt-image-2-image-to-image",
    "bytedance/v1-pro-fast-image-to-video",
    "elevenlabs/text-to-speech-multilingual-v2",
    "kling/ai-avatar-standard",
    "kling/ai-avatar-pro",
}


def write(p, obj):
    p = Path(p)
    p.parent.mkdir(parents=True, exist_ok=True)
    p.write_text(json.dumps(obj, ensure_ascii=False, indent=2))


def check(obj):
    if obj.get("code") != 200:
        raise RuntimeError(
            "Kie code=" + str(obj.get("code")) + " msg=" + str(obj.get("msg"))
        )
    return obj.get("data")


def auth():
    key = os.environ.get("KIE_API_KEY")
    if not key:
        raise RuntimeError("Falta KIE_API_KEY en .env.local o entorno")
    return {"Authorization": "Bearer " + key}


def approval(path, payload):
    x = json.loads(Path(path).read_text())
    for k in [
        "approved_by",
        "approved_at",
        "authorization_text",
        "scope",
        "max_credits",
        "max_usd",
    ]:
        if k not in x:
            raise RuntimeError("Aprobación incompleta: " + k)
    if not x.get("approved") or not x["authorization_text"]:
        raise RuntimeError("Falta aprobación explícita")
    if payload["model"] not in x.get("allowed_models", []):
        raise RuntimeError("Modelo fuera de aprobación")
    if x["max_credits"] <= 0 or x["max_usd"] <= 0:
        raise RuntimeError("Tope inválido")
    return x


# El control del presupuesto real pertenece a Claude; el API no reserva ese tope.
def submit(payload_path, approval_path, record):
```

```
    record = Path(record)
    if record.exists():
        raise RuntimeError(
            "Registro existente: consultar/reanudar; no crear otra tarea"
        )
    payload = json.loads(Path(payload_path).read_text())
    if payload.get("model") not in MODELS:
        raise RuntimeError("Modelo no admitido por este SOP")
    approval(approval_path, payload)
    write(
        record,
        {
            "status": "submitting",
            "payload_file": str(payload_path),
            "approval_file": str(approval_path),
            "model": payload["model"],
            "at": time.time(),
        },
    )
    try:
        r = requests.post(
            API + "/api/v1/jobs/createTask", headers=auth(), json=payload, timeout=120
        )
        r.raise_for_status()
        j = r.json()
        data = check(j)
        if not data or not data.get("taskId"):
            raise RuntimeError("Kie no devolvió taskId")
    except Exception:
        # No borrar registro: un timeout puede haber creado una tarea cobrada.
        raise RuntimeError(
            "Envío no confirmado. Revisar historial Kie antes de reenviar."
        ) from None
    write(
        record,
        {
            "status": "created",
            "taskId": data["taskId"],
            "model": payload["model"],
            "payload_file": str(payload_path),
            "approval_file": str(approval_path),
            "response": j,
        },
    )
    print("taskId=" + data["taskId"])


def poll(record, wait):
    path = Path(record)
    local = json.loads(path.read_text())
    task = local.get("taskId")
    if not task:
        raise RuntimeError("Sin taskId: reconciliar en historial del proveedor")
    deadline = time.monotonic() + wait
    while True:
        r = requests.get(
            API + "/api/v1/jobs/recordInfo",
            headers=auth(),
            params={"taskId": task},
            timeout=60,
        )
        r.raise_for_status()
        j = r.json()
        data = check(j) or {}
        local["last_response"] = j
        write(path, local)
```

```
        state = data.get("state")
        print("state=" + str(state))
        if state == "fail":
            raise RuntimeError(str(data.get("failMsg", "Fallo generación")))
        if state == "success":
            result = data.get("resultJson") or {}
            result = json.loads(result) if isinstance(result, str) else result
            urls = result.get("resultUrls", [])
            if not urls:
                raise RuntimeError("Success sin resultUrls; revisar respuesta guardada")
            local.update(
                status="success",
                result_urls=urls,
                creditsConsumed=data.get("creditsConsumed"),
            )
            write(path, local)
            return
        if time.monotonic() >= deadline:
            return
        time.sleep(min(10, max(0, deadline - time.monotonic())))


def download(record, out):
    local = json.loads(Path(record).read_text())
    url = local.get("result_urls", [None])[0]
    if not url:
        raise RuntimeError("Consultar tarea hasta success antes de descargar")
    dest = Path(out)
    dest.parent.mkdir(parents=True, exist_ok=True)
    tmp = dest.with_suffix(dest.suffix + ".part")
    # Resultado CDN: sin Authorization para no filtrar la API key a otro host.
    with requests.get(url, stream=True, timeout=120) as r:
        r.raise_for_status()
        if "text/html" in r.headers.get("content-type", ""):
            raise RuntimeError("CDN devolvió HTML")
        with tmp.open("wb") as f:
            for chunk in r.iter_content(1024 * 1024):
                f.write(chunk)
    if not tmp.stat().st_size:
        raise RuntimeError("Archivo vacío")
    tmp.replace(dest)
    print(str(dest))


def upload(file, record):
    f = Path(file)
    with f.open("rb") as stream:
        r = requests.post(
            UPLOAD + "/api/file-stream-upload",
            headers=auth(),
            files={"file": (f.name, stream)},
            data={"uploadPath": "ads/sop", "fileName": uuid.uuid4().hex + f.suffix},
            timeout=120,
        )
    r.raise_for_status()
    j = r.json()
    d = check(j)
    if not j.get("success") or not d.get("downloadUrl"):
        raise RuntimeError("Upload sin downloadUrl")
    write(record, j)
    print(d["downloadUrl"])


def main():
    p = argparse.ArgumentParser()
    p.add_argument("--env", default=".env.local")
```

```
    s = p.add_subparsers(dest="cmd", required=True)
    s.add_parser("credits")
    t = s.add_parser("submit")
    t.add_argument("payload")
    t.add_argument("--approval", required=True)
    t.add_argument("--record", required=True)
    t = s.add_parser("poll")
    t.add_argument("record")
    t.add_argument("--wait", type=int, default=0)
    t = s.add_parser("download")
    t.add_argument("record")
    t.add_argument("out")
    t = s.add_parser("upload")
    t.add_argument("file")
    t.add_argument("--record", required=True)
    a = p.parse_args()
    load_dotenv(a.env)
    if a.cmd == "credits":
        r = requests.get(API + "/api/v1/chat/credit", headers=auth(), timeout=60)
        r.raise_for_status()
        print(json.dumps(check(r.json())))
    elif a.cmd == "submit":
        submit(a.payload, a.approval, a.record)
    elif a.cmd == "poll":
        if a.wait < 0 or a.wait > 50:
            raise RuntimeError("Usar --wait entre 0 y 50 y reanudar después")
        poll(a.record, a.wait)
    elif a.cmd == "download":
        download(a.record, a.out)
    elif a.cmd == "upload":
        upload(a.file, a.record)


if __name__ == "__main__":
    main()
```

Archivo: ad-creator/SKILL.md

```
---
name: ad-creator
description: Produce anuncios fieles y dinámicos desde Watch con Google Chrome, SSSTik, REST Kie y HyperFrames; controla presupuesto ad1 y cinco adicionales.
---

Leer references/sop-anuncios.md y references/sop-claude-exacto.md antes de producir.
Aplicar todos sus pasos de Chrome/TikTok/Pinterest/SSSTik, carpetas, scripts API,
modelos y QC. Watch analiza todos los clips; esta skill obtiene recursos y produce.
Presupuesto ad1 aprobado, ad1 terminado para revisión, presupuesto incremental
aprobado de cinco y tanda de seis. No publicar campañas.
Para VSL existente leer references/vsl-encuadre-original.md; para formatos
presentador/sketch/cartoon leer references/formatos-aprobados.md; para materiales
leer references/edicion-dinamica-materiales.md. Preservar guion íntegro y encuadre
original de narradora. No confundir aprobación creativa con ventas.
Scripts: iniciar-proyecto.py crea árbol/CSV; kie.py realiza credits, upload,
submit, poll y download. Leer su contrato y registrar autorización antes de submit.
```

Archivo: watch/SKILL.md

```
---
name: watch
description: Analiza exhaustivamente clips, formatos, movimiento y audio de referencias para reconstrucción literal en el proceso de modelado.
---

Leer ../ad-creator/references/sop-anuncios.md y capítulos 05-06 de
../ad-creator/references/sop-claude-exacto.md. Descargar/observar referencia en
Google Chrome con Claude in Chrome; no reemplazar percepción por transcripción.
Analizar todos los clips y formatos, incluidos insertos, capas y split-screen.
Cada ID lleva tiempos, objeto literal, acción por fases, encuadre/cámara/luz,
voz, música/SFX, captions/popups/transiciones e imprescindibles para reconstruir.
Entregar informe y clips.csv, guion íntegro y límites. Si hay papa, documentar
papa y su acción concreta. No generar assets ni anuncios; pasar mapa a ad-creator.
```

Archivo: modeler/SKILL.md

```
---
name: modeler
description: Orquesta la fase de creativos fieles y dinámicos con Watch y ad-creator en Claude.
---

Orquestador mínimo de anuncios. Leer ../ad-creator/references/sop-anuncios.md y
../ad-creator/references/sop-claude-exacto.md. Obtener oferta, referencia y mercado
existentes; invocar Watch exhaustivo y luego ad-creator con su mapa literal.
Exigir pasos exactos de Google Chrome/SSSTik, sourcing primero internet/TikTok/
Pinterest, carpetas, Kie para faltantes/UGC y voz/música/HyperFrames.
Seis totales en dos tramos con presupuesto aprobado y revisión ad1.
Actualizar panel/entrega y registrar feedback. No publicar campañas.
Si existe modeler/modelar completo, integrar esta fase sin sustituir sus otras fases.
```

Referencias que debe crear Claude al instalar solo desde PDF

Guardar la primera parte como ad-creator/references/sop-claude-exacto.md, el contrato integral como sop-anuncios.md y las tres recetas completas con sus nombres vsl-encuadre-original.md, formatos-aprobados.md y edicion-dinamica-materiales.md en ese mismo directorio. Así quedan resueltos todos los enlaces de SKILL.md. No guardar el anexo dentro de cada referencia.